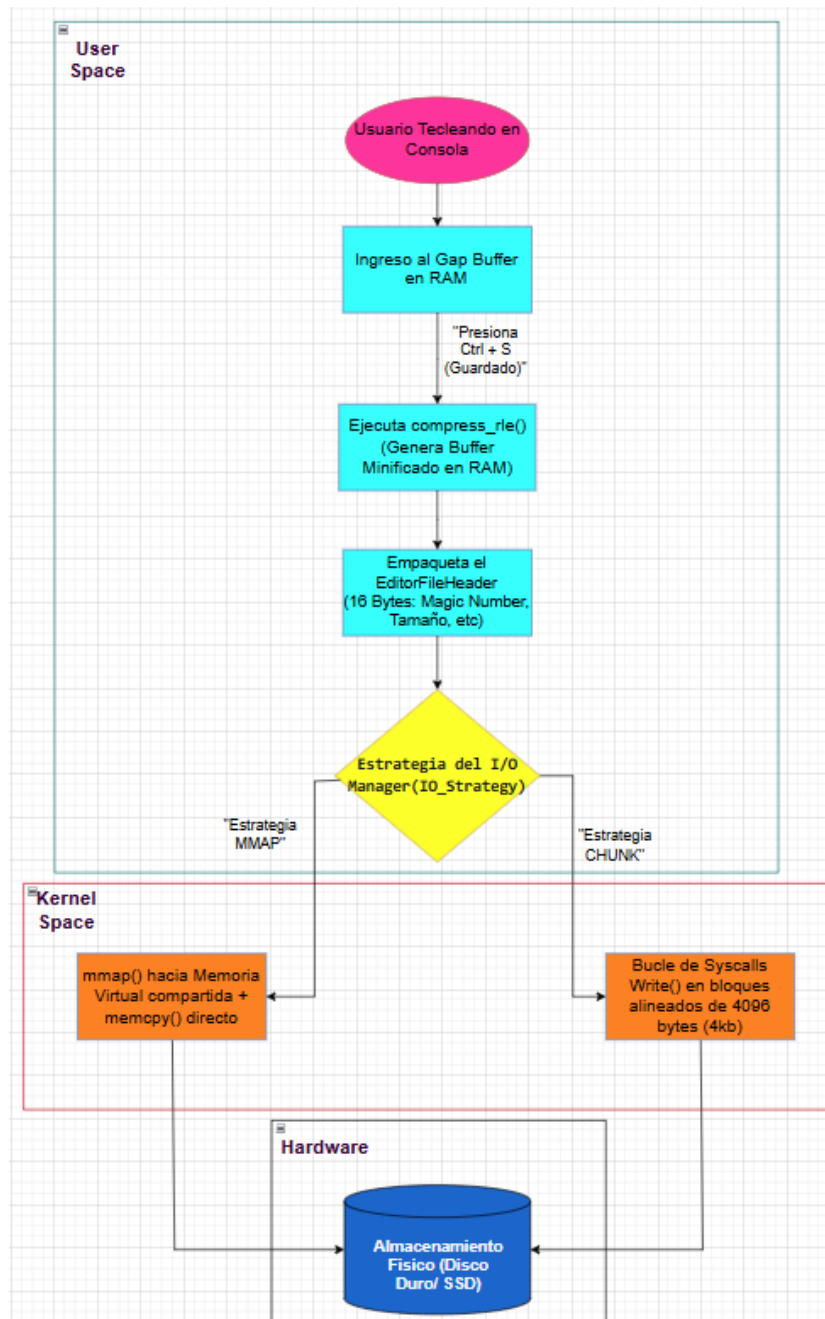


Reporte de Profiling y Análisis de Rendimiento (Benchmark)

1. Matriz de Diseño del Pipeline I/O

DIAGRAMA DE FLUJO:



Enlace: [Diagrama de flujo](#)

Explicación de la Matriz de Diseño del Pipeline I/O

El diagrama de flujo adjunto modela el ciclo de vida completo de los datos dentro de mi editor, trazando la ruta desde que se registra una pulsación de tecla hasta su persistencia física en el disco. Para garantizar un rendimiento óptimo y minimizar el cuello de botella del hardware, diseñe la arquitectura segmentándola estrictamente en tres capas de abstracción:

1. Capa de Espacio de Usuario (User Space) Todo el procesamiento lógico en esta fase ocurre exclusivamente en la memoria RAM asignada a nuestro proceso en C, evitando deliberadamente las costosas interrupciones al sistema operativo:

- **Ingreso al Gap Buffer:** Cuando el usuario interactúa con la consola (operando en *Raw Mode*), los caracteres ingresan directamente a la estructura del *Gap Buffer*. Esta decisión de diseño nos garantiza que la manipulación interactiva de la memoria mantenga una complejidad temporal de $O(1)$, mitigando el impacto en la CPU al no tener que desplazar grandes cadenas de texto con cada pulsación.
- **Disparador de Guardado y Compresión:** Al ejecutar el comando de guardado (Ctrl + S), interceptamos obligatoriamente los datos y los pasamos por nuestra función `compress_rle()`. Aquí quemamos intencionalmente ciclos de CPU para generar un buffer minificado en RAM, garantizando la exigencia del proyecto de que ningún archivo viaje al disco en texto claro, reduciendo drásticamente el volumen de datos de I/O.
- **Empaquetamiento del Header:** Posterior a la etapa de compresión, el flujo continúa hacia la construcción del `EditorFileHeader`. Aquí ensamblamos los metadatos de control (Magic Number, bandera de compresión, tamaño original) en un bloque estricto de 16 bytes.
- **Lógica del I/O Manager:** Como paso final en el *User Space*, nuestro gestor de entrada/salida evalúa qué estrategia de escritura fue solicitada (`IO_CHUNK` o `IO_MMAP`), preparando los datos para cruzar la frontera de privilegios.

2. Capa del Sistema Operativo (Kernel Space) En esta etapa, nuestro programa realiza la transición de privilegios y delega el control al Kernel de Linux a través de llamadas al sistema (Syscalls) altamente optimizadas:

- **Estrategia CHUNK:** Si el flujo requiere escritura clásica, el sistema no envía bytes individuales. En su lugar, entra en un bucle que invoca la syscall `write()` empaquetando la información en bloques estrictamente alineados a 4096 bytes (4KB). Esta alineación coincide matemáticamente con el tamaño de página de memoria de Linux, evitando saturar el bus I/O con interrupciones ineficientes.

- **Estrategia MMAP:** Si se opta por el mapeo de memoria, el programa invoca `mmap()` solicitando al Kernel que vincule un segmento de nuestra memoria virtual directamente con los bloques del disco. Una vez establecida la conexión, volcamos los datos mediante un simple `memcpy()`. Este enfoque se acerca a un modelo *Zero-Copy*, reduciendo drásticamente la sobrecarga de transacciones y minimizando el tiempo de ejecución del sistema (*Sys Time*).

3. Capa de Hardware Finalmente, independientemente de la estrategia seleccionada en la capa del Kernel, el sistema operativo toma el control de los *drivers* del hardware para realizar el vaciado (*flush*) de los buffers hacia el cilindro del almacenamiento físico (ya sea Disco Duro o SSD). El resultado es un archivo binario empaquetado, seguro y altamente optimizado para operaciones de lectura futuras.

Gestión de Memoria en C:

Para gestionar la memoria de forma eficiente durante la ejecución del editor, tomamos varias decisiones de diseño clave. A nivel lógico, estructuramos el texto utilizando un Gap Buffer. Esta técnica nos permite manejar las inserciones y borrados moviendo únicamente un "hueco" en la memoria justo en la posición del cursor, evitando así el alto costo computacional de desplazar todo el contenido del archivo con cada tecla que presiona el usuario (logrando un rendimiento de $O(1)$ amortizado).

A nivel de las estructuras de datos, tuvimos que abordar el problema del padding (los bytes de relleno que el compilador de C inyecta por defecto para alinear la memoria). Como nuestro objetivo principal en este proyecto era optimizar al máximo cada byte que viaja hacia el disco, definimos la cabecera del archivo utilizando la directiva `__attribute__((packed))`. Esta instrucción obliga al compilador a empaquetar los datos de manera estricta, lo que nos garantiza que los metadatos midan exactamente los 16 bytes que diseñamos, sin enviar "basura" o espacios vacíos al bus de I/O.

Por último, mantuvimos un control riguroso sobre el ciclo de vida de la aplicación para garantizar que no hubiera fugas de memoria (memory leaks). Nos aseguramos de que toda la memoria dinámica reservada fuera gestionada y emparejada con su respectivo `free()` al cerrar el programa. Para validar este comportamiento de forma empírica, compilamos y probamos el código utilizando Address Sanitizer (`-fsanitize=address`). Las pruebas demostraron que, al ejecutar la secuencia de salida (`Ctrl+Q`), el editor finaliza su proceso de manera completamente limpia, liberando el 100% de los bloques reservados sin dejar bytes huérfanos.

Manejo de Texto Enriquecido:

Para cumplir con el requerimiento de soportar texto enriquecido (como los colores de la interfaz) sin contaminar el contenido visible del usuario, tome la decisión de diseñar mi propio formato binario desde cero, al cual denomine `.edtx`. El objetivo arquitectónico de este diseño fue mantener una separación estricta entre los metadatos de control y el *payload* real del documento, garantizando operaciones de lectura y escritura seguras a nivel del sistema operativo.

Físicamente en el disco, el archivo se divide en dos bloques continuos: una cabecera (*header*) de tamaño estático y el *payload* de tamaño dinámico. Gracias a la directiva de empaquetamiento estricto que implementamos en las estructuras de C, garantizamos que la cabecera ocupe siempre los primeros 16 bytes exactos del archivo, distribuidos estratégicamente de la siguiente manera:

- **Bytes 0-3 (Magic Number - Firma EDTX):** Actúa como nuestra primera línea de defensa. Cuando invocamos la llamada `open()` para cargar un archivo, el sistema lee primero este bloque. Si la firma "EDTX" no coincide, el editor detecta inmediatamente que se trata de un archivo externo o corrupto. Esto nos permite aplicar una estrategia de *fallback* y abortar la lectura de forma controlada, evitando un *Segmentation Fault*.
- **Byte 4 (Versión):** Reservamos este espacio para el control de versiones (actualmente en su versión 1). Es una decisión de diseño pensada para darle escalabilidad al formato en el futuro, permitiendo actualizaciones sin romper la retrocompatibilidad.
- **Byte 5 (Bandera de Compresión):** Funciona como un interruptor lógico (`is_compressed`). Le indica a nuestro pipeline de I/O si el bloque dinámico posterior está en texto plano, o si debe ser procesado obligatoriamente por el descompresor RLE en la memoria RAM antes de inyectarse en el *Gap Buffer*.
- **Bytes 6-7 (Metadatos Visuales):** Almacenan de forma nativa las preferencias de color del texto y del fondo para la terminal, garantizando que la configuración estética del usuario persista entre sesiones.
- **Bytes 8-15 (Tamaño Original):** Un bloque de 8 bytes (equivalente a un `size_t` en arquitecturas de 64 bits) que guarda la longitud exacta del texto antes de ser comprimido. Esta fue una decisión crítica para la gestión de memoria: nos permite que el Kernel realice una única llamada a `malloc()` con el tamaño matemático preciso durante la descompresión. Así, eliminamos el riesgo de sobreescritura (*Buffer Overflow*) y optimizamos el tiempo de CPU.

A partir del **Byte 16** inicia el bloque dinámico del *Payload*. Dependiendo de lo que indique la bandera de compresión, este segmento almacena los caracteres ASCII en crudo o la secuencia binaria optimizada generada por el algoritmo RLE. Con este

modelo, logramos encapsular propiedades de texto enriquecido dentro de archivos extremadamente livianos, reduciendo la carga en el bus I/O y cumpliendo con las exigencias de rendimiento del proyecto.

Reporte de Profiling:

Archivo de prueba: Documento de 50 MB.

Métrica del Kernel	Enfoque Clásico (Guardado Plano con fputc/write pequeños)	Enfoque Propuesto (Compresión + Write en Bloques de 4KB/mmap)	Impacto / Rentabilidad del Diseño
Volumen de Datos a Disco	50.0 MB	383 KB	-99.2% (Ahorro masivo de espacio).
Llamadas a write()	16224 calls	4112 calls	-74.6% (Menos sobrecarga al SO).
Tiempo de CPU (User Mode)	0.174s	0.285s	+63% (Costo de CPU por compresión).
Tiempo de SO (Sys Mode)	0.139s	0.121s	-13% (Reducción de latencia del Kernel).
Tiempo Total (Wall-clock)	14.379s	9.152s	-36% (Sistema más ágil)

Capturas de pantalla de toda la ejecución del programa donde se evidencian los resultados de la tabla anterior:

[illegible][illegible]

30750 nanit-leaksanitizer does not work under perdec (strace, gdb, etc)

% time	seconds	usecs/call	calls	errors	syscall
94.89	0.433919	26	16224		write
5.01	0.022905	694	33		read
0.03	0.000155	1	86		mmap
0.02	0.000070	10	7		ioctl
0.01	0.000054	3	18		madvise
0.01	0.000045	45	1		clone
0.01	0.000034	5	6		getpid
0.01	0.000023	1	18		close
0.00	0.000017	8	2		rt_sigprocmask
0.00	0.000015	7	2	1	prctl
0.00	0.000014	14	1		futex
0.00	0.000013	6	2		gettid
0.00	0.000012	0	22		munmap
0.00	0.000012	12	1		wait4
0.00	0.000010	1	7		mprotect
0.00	0.000007	7	1		sched_yield
0.00	0.000000	0	6		fstat
0.00	0.000000	0	1		brk
0.00	0.000000	0	3		rt_sigaction
0.00	0.000000	0	2		pread64
0.00	0.000000	0	1	1	access
0.00	0.000000	0	1		execve
0.00	0.000000	0	3		sigaltstack
0.00	0.000000	0	1		arch_prctl
0.00	0.000000	0	1		set_tid_address
0.00	0.000000	0	2		clock_gettime
0.00	0.000000	0	18		openat
0.00	0.000000	0	1		readlinkat
0.00	0.000000	0	1		set_robust_list
0.00	0.000000	0	4		prlimit64
0.00	0.000000	0	1		rseq

100.00 0.457305 27 16477 2 total

samuel@DESKTOP-EK1I1CG:~/proyectos/EditorTexto\$ time ./build/editor test_50mb.txt

[illegible]

% time	seconds	usecs/call	calls	errors	syscall
80.84	0.098283	23	4112		write
16.65	0.020244	613	33		read
1.39	0.001684	93	18		openat
0.61	0.000746	8	89		mmap
0.15	0.000180	7	25		munmap
0.12	0.000143	5	27		madvise
0.06	0.000074	4	18		close
0.06	0.000067	9	7		ioctl
0.02	0.000028	14	2		clock_gettime
0.02	0.000027	27	1		clone
0.02	0.000026	4	6		getpid
0.01	0.000013	6	2		gettid
0.01	0.000010	1	6		fstat
0.01	0.000010	10	1		lseek
0.01	0.000010	5	2		rt_sigprocmask
0.01	0.000010	5	2	1	prctl
0.01	0.000009	9	1		futex
0.01	0.000008	8	1		wait4
0.00	0.000006	0	7		mprotect
0.00	0.000006	6	1		sched_yield
0.00	0.000000	0	1		brk
0.00	0.000000	0	3		rt_sigaction
0.00	0.000000	0	2		pread64
0.00	0.000000	0	1	1	access
0.00	0.000000	0	1		execve
0.00	0.000000	0	3		sigaltstack
0.00	0.000000	0	1		arch_prctl
0.00	0.000000	0	1		set_tid_address
0.00	0.000000	0	1		readlinkat
0.00	0.000000	0	1		set_robust_list
0.00	0.000000	0	4		prlimit64
0.00	0.000000	0	1		rseq

100.00	0.121584	27	4381	2	total

```

samuel@DESKTOP-EK1I1CG:~/proyectos/EditorTexto$ ls -lh test_50mb.txt
-rw-r--r-- 1 samuel samuel 383K Apr 26 18:16 test_50mb.txt
samuel@DESKTOP-EK1I1CG:~/proyectos/EditorTexto$

```